

SYSTEM ARCHITECTURE DOCUMENT Real-Time Weather Data Platform

1. Overview

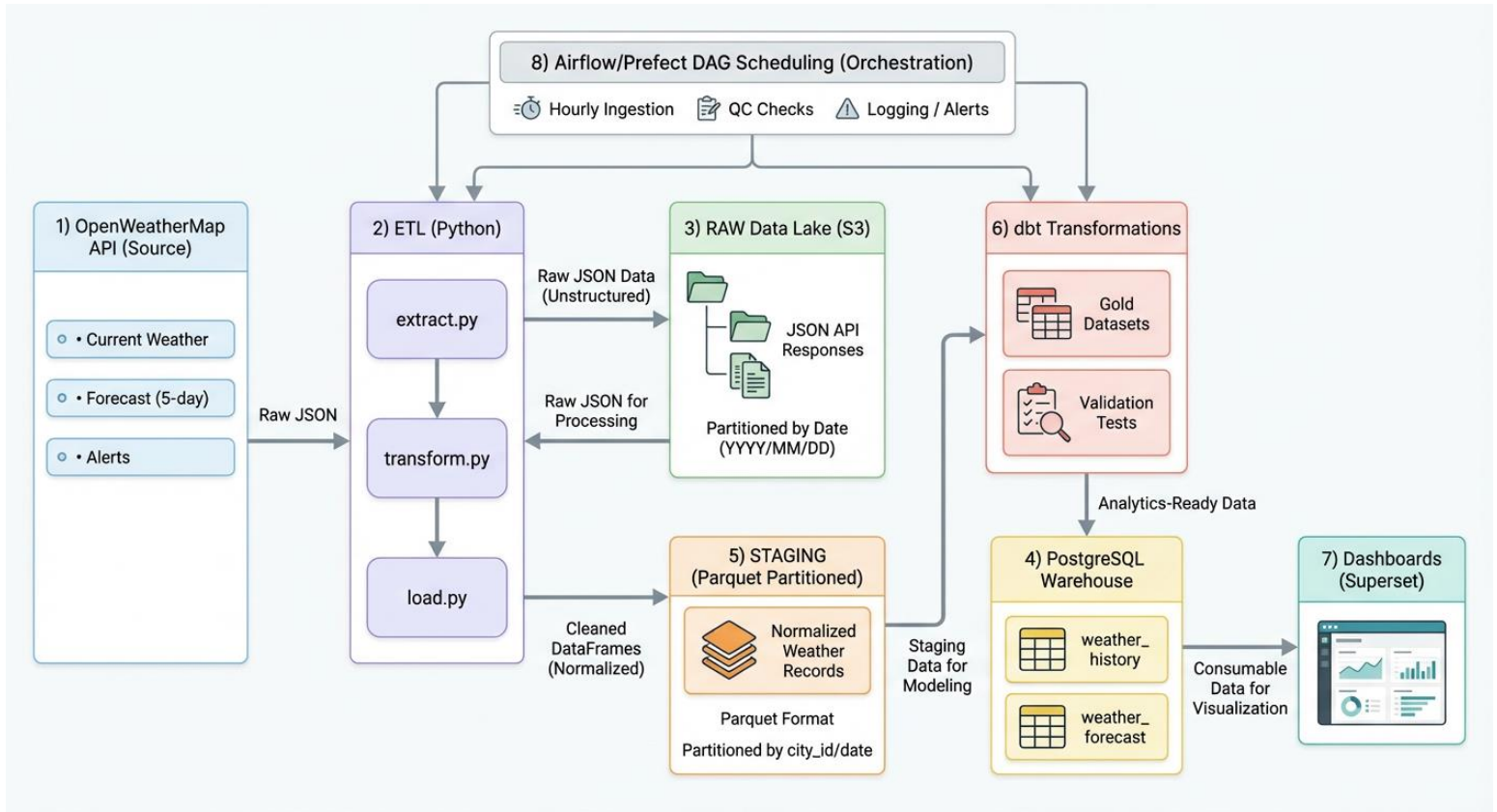
This system is a real-time data pipeline that ingests weather data from the OpenWeatherMap API, processes and transforms it using Python, and stores both raw and cleaned weather data in a cloud-hosted PostgreSQL database and an S3-compatible data lake.

The platform is designed following modern data engineering principles:

- Modular ingestion
- Scalable transformations
- Lakehouse pattern (Raw → Staging → Analytics)
- Orchestration with Airflow/Prefect
- Data quality validation
- Automated scheduling
- Separation of compute, storage, and orchestration

This architecture supports **multiple data sources, multi-city ingestion, historical tracking, analytics, and dashboarding** while being deployable on free-tier cloud services.

2. High-Level Architecture Diagram



3. Functional Requirements

Extraction

- Fetch weather data for **multiple cities**
- Pull:
 - Current weather
 - 5-day / 3-hour forecast
 - Weather alerts
- Store raw JSON responses into the data lake

Transformation

- Normalize temperature, wind, and pressure units
- Convert timestamps to:
 - UTC
 - Local timezone for each city
- Validate required fields
- Flatten nested JSON fields (alerts)

- Produce clean, denormalized analytical tables

Loading

- Insert cleaned data into PostgreSQL in:
 - weather_history
 - weather_forecast
 - weather_alerts
- Use bulk inserts or COPY for efficiency
- Maintain historical records (no overwrites)

Orchestration

- Hourly DAG triggered by Airflow or Prefect
- Retry on API failures
- Notify on pipeline failure
- Log run metadata (start/end time, rows processed)

Monitoring

- Detect missing fields from API
- Detect stale data (if API fails for many hours)
- Validate temperature ranges (sanity checks)
- Detect schema drift
- Record pipeline metrics:
 - API latency
 - Rows inserted
 - Error counts

4. Non-Functional Requirements

Requirement	Description
Scalability	Can support dozens of cities; Parquet partitions support large volumes
Data Quality	Must be validated with automated tests
Reliability	Retries + logging ensure robust runs
Cost Efficiency	Built entirely using free-tier components
Extensibility	New data sources or cities can be added without redesign

5. Components Detail

5.1 Extract Layer

Module: extract.py

Features:

- Query API endpoints for:
 - /weather
 - /forecast
 - /alerts (inside OneCall API)
 - Parameterized by city list
 - Logs latency, status codes, and payload size
 - Writes raw responses to:
 - /data_lake/raw/YYYY/MM/DD/city/*.json
-

5.2 Transform Layer

Module: transform.py

Transforms include:

- Flattening nested objects (main, wind, weather[0])
 - Converting:
 - Kelvin → Celsius
 - UTC timestamps
 - Creating unique primary keys
 - Extracting alert severity + event type
 - Handling missing fields
 - Creating Parquet partitions:
 - /data_lake/staging/city=Victoria/year=2025/month=12/day=04/*.parquet
-

5.3 Load Layer

Module: load.py

Loads data into PostgreSQL using:

- psycopg2 COPY
- Bulk inserts

- Staged loads to avoid partial inserts
- Error catching + rollback

Tables:

- weather_history
 - weather_forecast
 - weather_alerts
 - cities (city metadata table)
-

5.4 Orchestration Layer

Using **Airflow** or **Prefect**:

DAG Tasks:

1. extract_current_weather
2. extract_forecast
3. extract_alerts
4. validate_raw_data
5. transform_staging
6. load_database
7. run_data_quality_checks
8. archive_run_logs

Scheduling:

- Run every **60 minutes**
 - Retry 3 times on failure
 - Send notifications (email/Slack webhook)
-

5.5 Data Lake Layer

Format:

- Raw JSON
- Cleaned Parquet

Benefits:

- Faster analytics

- Cheap storage
 - Partitioned for scalable queries
 - Decouples processing from database
-

5.6 Data Quality Layer

Using:

- Custom Python validations
or
- Great Expectations

Checks:

- Missing or null values
 - API drift (unexpected fields)
 - Value range checks
 - Duplicate timestamps
 - Row count anomalies
 - Time gaps in ingestion
-

6. Future Extensions

- Near-real-time (every 5 minutes) ingestion
- Adding radar/satellite data
- Geospatial analysis with PostGIS
- Anomaly detection ML model
- Deploying public API for weather history